

Web Publishing Project: Coca-Cola

코카-콜라 웹사이트 구현



레츠기니

목차



01. 팀원 소개

02. 주제

03. 사용한 툴

04. 주요 기능 구현

05. 시연

06. 후기

01. 팀 - 레츠기니



오승찬

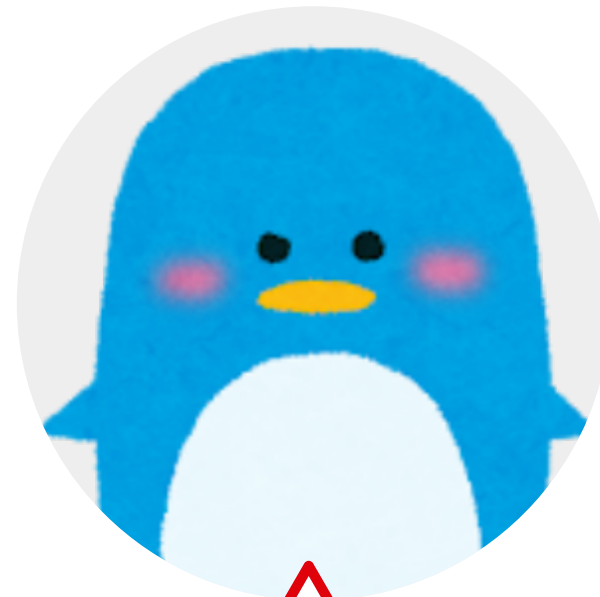
팀장

Sustainability



김다원

Brands, 전체 파일 종합
아이디/비밀번호 찾기



김수진

Discover(Opener, Creation,
Cokeplay App)



이나영

Main, Navigation,
Our Company (Coca-Cola
Company ,Since 1886)



장소예

Discover(Brand story, Coke
& meal, Originals), Footer,
로그인, 회원가입

02. 주제 - Coca Cola



코카콜라 사이트는 코카콜라라는 브랜드가 주는 '짜릿함과 즐거움'을 사용자가 직접 체험하고
소통하게 만드는 디지털 홍보 공간입니다

03. 사용한 툴



Spring



sql Developer

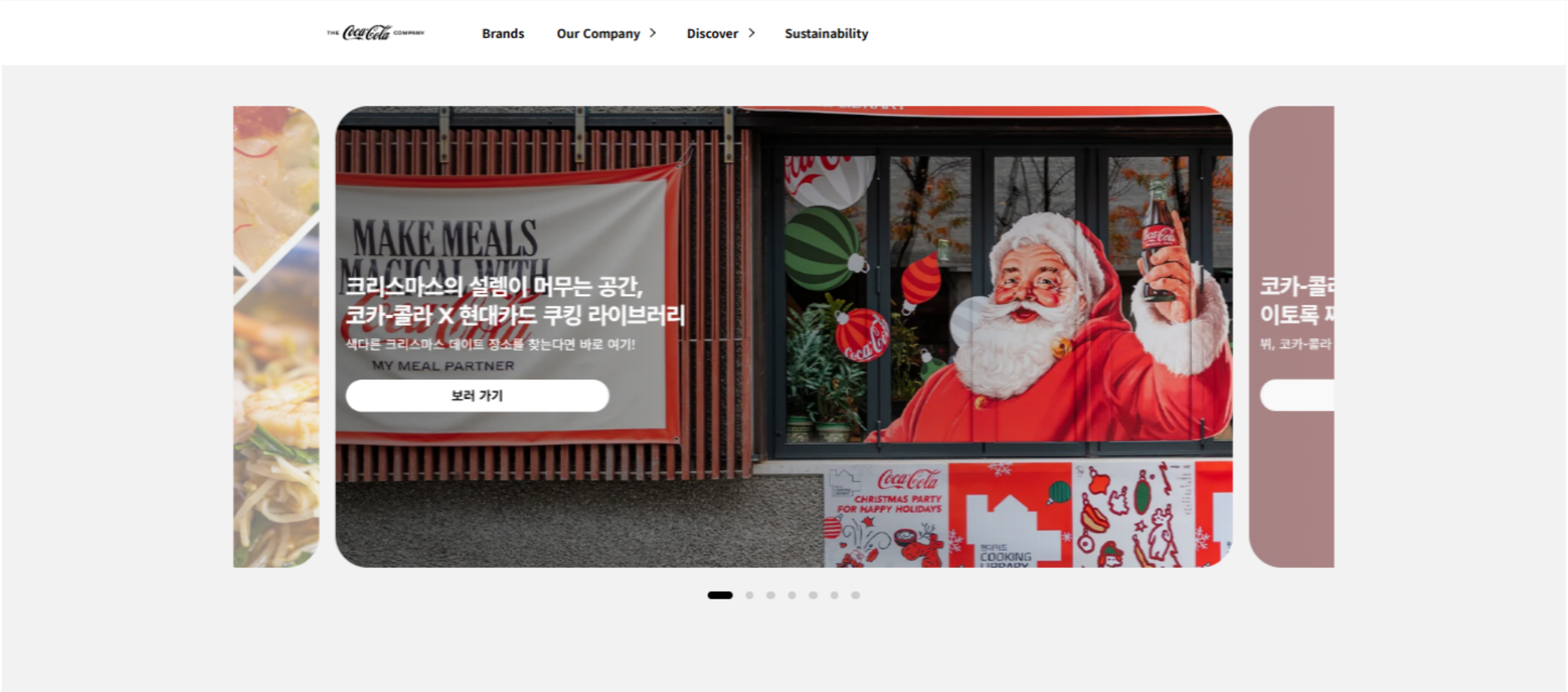


GitHub



Notion

04. 주요 기능 구현 - Main



04. 주요 기능 구현 - Main

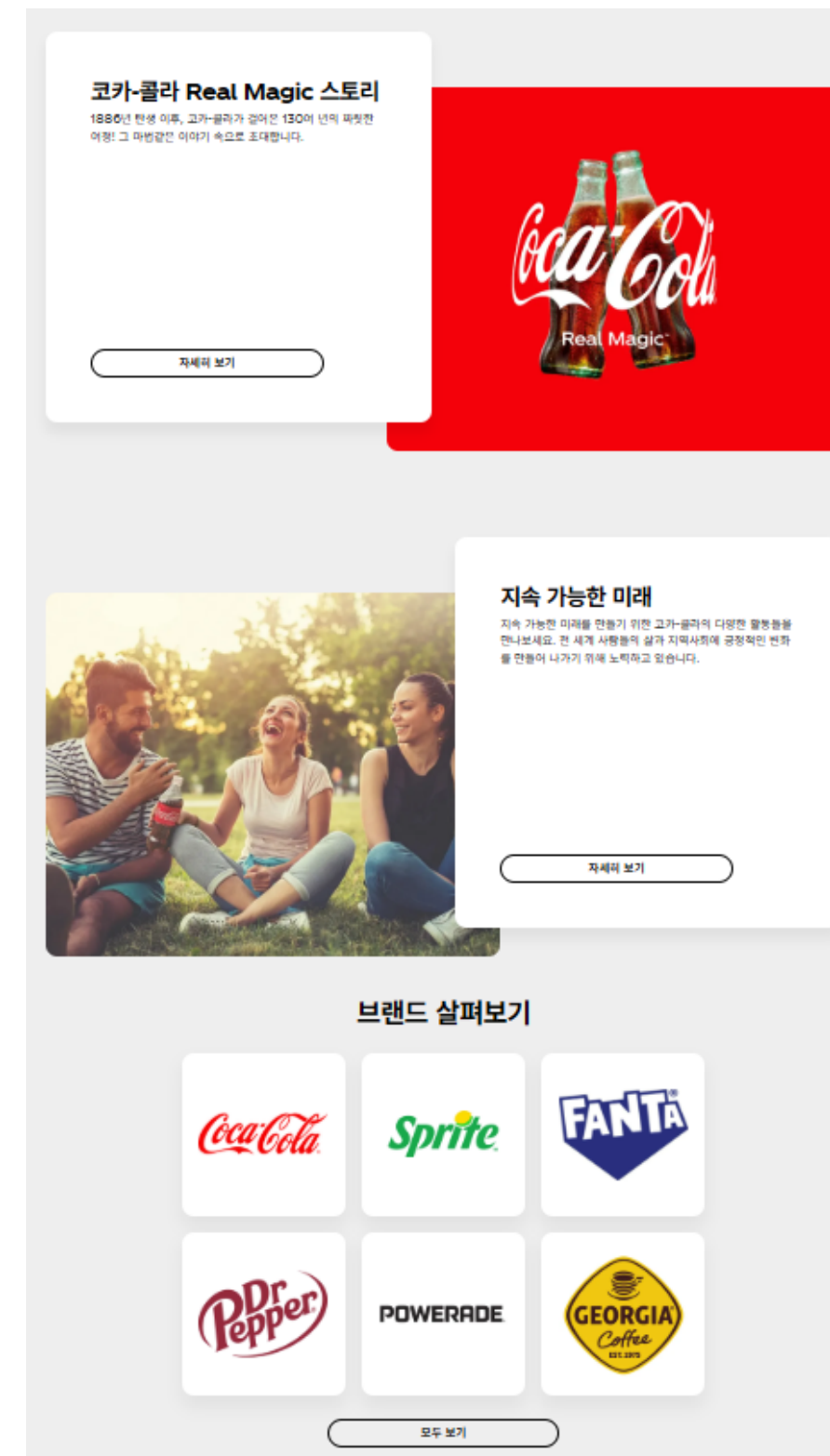
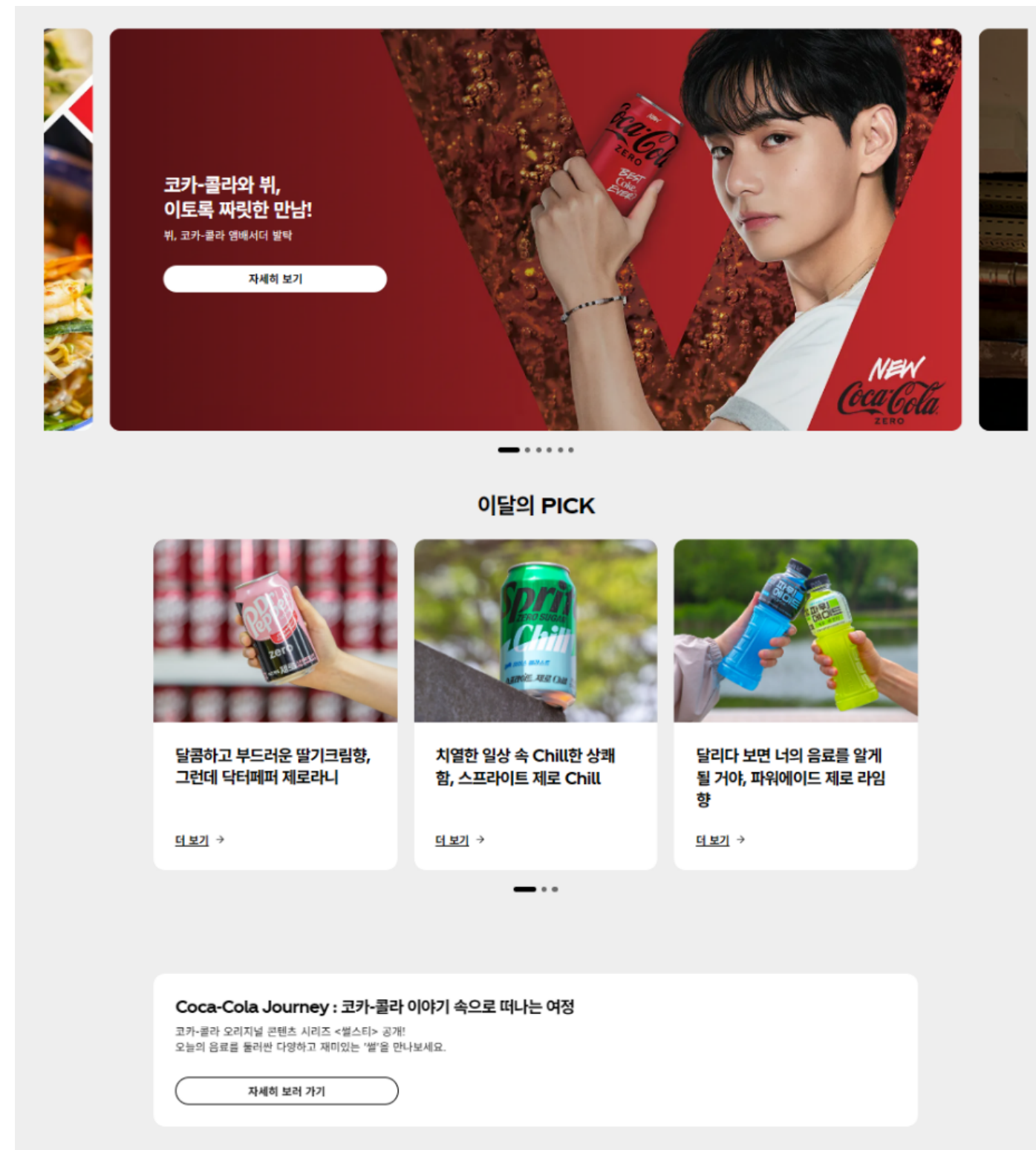
JSTL의 `<c:forEach>` 문을 사용하여 서버로부터 전달받은 데이터를 반복 출력.
이미지 경로 설정 시 `${pageContext.request.contextPath}`를 활용

```
<main>
  <!-- [1] 키이미지 슬라이드 -->
  <section class="slider-section">
    <div class="swiper mySwiper">
      <div class="swiper-wrapper">
        <c:forEach var="i" begin="1" end="7">
          <div class="swiper-slide">
            
          </div>
        </c:forEach>
      </div>
    </div>
    <div class="dots-container">
      <div class="swiper-pagination my-pagination"></div>
    </div>
  </section>
```

외부 CDN을 통해 Swiper 라이브러리를 로드

```
<script
  src="https://cdn.jsdelivr.net/npm/swiper@11/swiper-bundle.min.js"></script>
<script>
//[1] 상단 메인 슬라이더 설정
const swiper = new Swiper(".mySwiper", {
  slidesPerView: "auto",
  centeredSlides: true,
  spaceBetween: 20,
  loop: true,
  initialSlide: 0,
  loopedSlides: 7,
  pagination: {
    el: ".my-pagination",
    clickable: true,
  },
  speed: 600,
  watchSlidesProgress: true,
  normalizeSlideIndex: true,
});
```


04. 주요 기능 구현 - Main



04. 주요 기능 구현 - Main

```
@Controller
@RequestMapping("/cocacola")
public class OurCompanyController {

    // 1. 기존 DAO 대신 Mapper 인터페이스를 주입받습니다.
    @Autowired
    private OurCompanyMapper ourCompanyMapper;

    @Autowired
    private BrandDAO brandDAO;

    @GetMapping("/ourcompany")
    public String process(
        @RequestParam(value = "page", defaultValue = "main") String page,
        Model model
    ) throws Exception {

        // 2. Mapper를 통해 데이터를 가져와 모델에 담습니다.
        // ourCompanyMapper.selectAllContents() 메서드는 XML의 id="selectAllContents"와 연결됩니다.
        model.addAttribute("items", ourCompanyMapper.selectAllContents());
        model.addAttribute("mainBrands", brandDAO.selectBrandList());

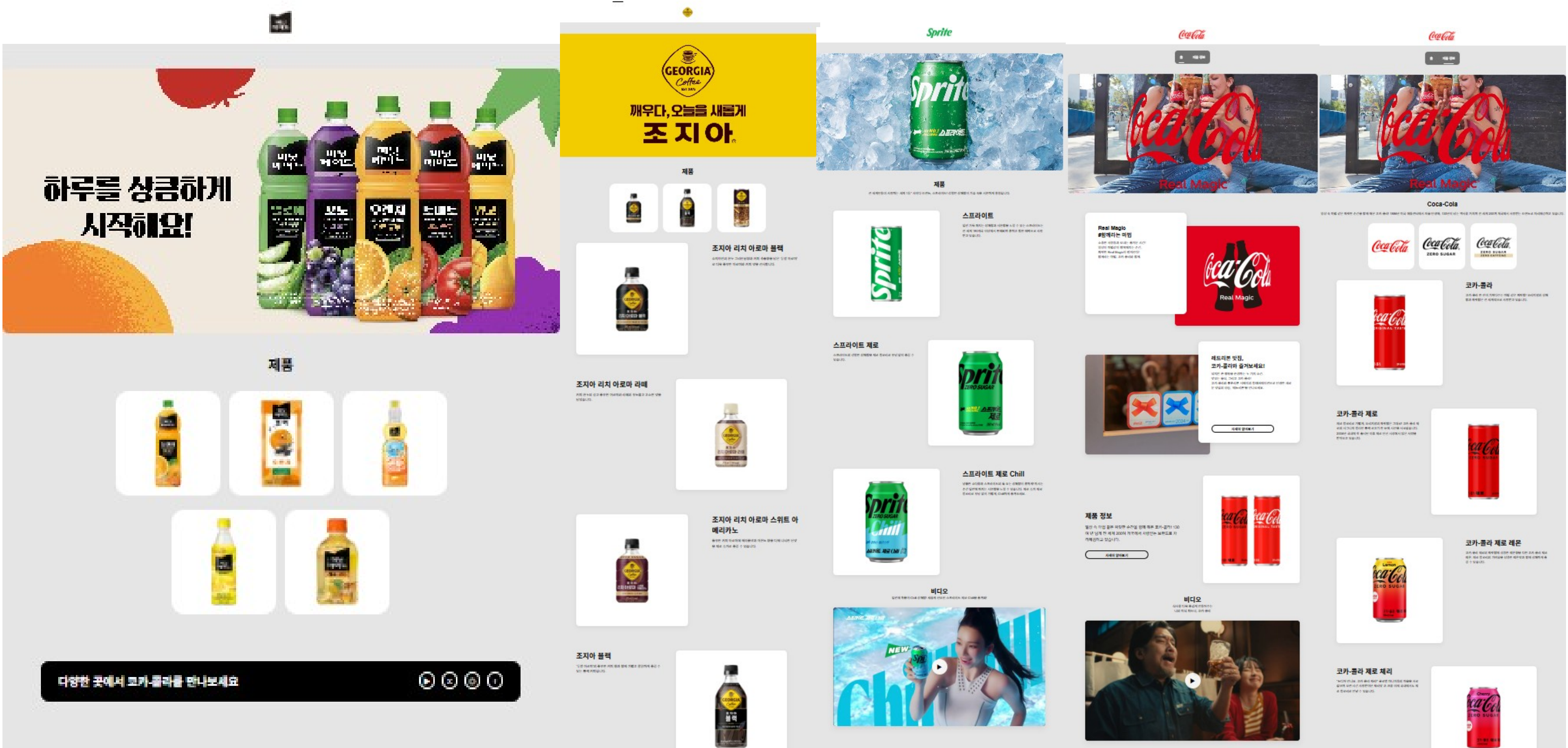
        // 3. 페이지 값에 따른 뷰 경로 리턴
        switch (page) {
            case "since":
                // WEB-INF/views/cocacola/ourcompany/since_01.jsp 로 이동
                return "cocacola/ourcompany/since_01";
            case "main":
                // WEB-INF/views/cocacola/ourcompany/main_01.jsp 로 이동
                return "cocacola/ourcompany/main_01";
            case "company":
                // URL 파라미터가 page=company일 때를 대비해 명시적으로 관리하면 좋습니다.
                return "cocacola/ourcompany/ourcompany_01";
            default:
                // 기본값: WEB-INF/views/cocacola/ourcompany/ourcompany_01.jsp
                return "cocacola/ourcompany/ourcompany_01";
        }
    }
}
```

OurCompanyController

@RequestParam을 통해 page 값을 주입받으며,
값이 없을 경우 'main'을 기본값으로 설정해
로직의 안정성을 확보합니다.

내부 switch 로직을 통해 요청에 부합하는 JSP 물리 경로
를 논리적으로 결정하여 리턴합니다.

04. 주요 기능 구현 - Brands



04. 주요 기능 구현 - Brands

```
package cocacola.mvc.controller;

import java.util.List;

@Controller
@RequestMapping("/cocacola")
public class BrandController {

    @Autowired
    private BrandMapper brandMapper; // 스프링이 자동으로 주입

    @GetMapping("/brands")
    public String process(Model model) throws Exception {

        // 1. 데이터 조회
        List<BrandDTO> list = brandMapper.selectBrandList();

        // 2. Model에 저장 (기존 request.setAttribute 대체)
        model.addAttribute("mainBrands", list);

        // 3. 뷰 리턴 (prefix/suffix에 의해 경로 완성)
        return "cocacola/brands/brandList";
    }
}
```

기존에는 .do 기반 컨트롤러에서
HttpServletRequest로 파라미터를 받고
DAO를 직접 생성해서 SQL을 실행하는 구조

@Controller는 요청을 처리하는 역할이고
@Autowired를 통해 Mapper를 주입받습니다.
DAO를 직접 생성하지 않고 Spring이 객체를 관리

.do 대신 @GetMapping으로 URL과 메서드를 매핑
요청 URL은 /cocacola/brands

DB 접근은 Mapper 인터페이스를 통해
이루어지며 실제 SQL은 XML에 분리

04. 주요 기능 구현 - Brands

```
import java.util.List;

@Controller
@RequestMapping("/cocacola")
public class BrandDetailController {

    @Autowired
    private BrandMapper brandMapper;

    @GetMapping("/brandDetail")
    public String process(
        @RequestParam("brandId") int brandId,
        @RequestParam(value = "tab", defaultValue = "home") String tab,
        Model model) throws Exception {

        BrandDTO brand = brandMapper.selectBrand(brandId);
        if (brand == null) {
            return "error/404";
        }

        BrandVideoDTO video = brandMapper.selectBrandVideo(brandId);
        if (video != null) {
            brand.setYoutubeVideoId(video.getYoutubeVideoId());
            brand.setVideoDescription(video.getVideoDescription());
            brand.setVideoThumbnailUrl(video.getVideoThumbnailUrl());
        }

        List<ProductDTO> products = brandMapper.selectProducts(brandId);
        brand.setProducts(products);

        if (brandId != 9) {
            List<SubBrandDTO> subBrands = brandMapper.selectSubBrands(brandId);
            model.addAttribute("subBrands", subBrands);
        }

        model.addAttribute("brand", brand);

        if (brandId == 1) {
            if ("products".equals(tab)) {
                return "cocacola/brands/cocacolaProducts";
            }
            return "cocacola/brands/cocacolaHomeAndProduct";
        } else if (brandId == 8) {
            return "cocacola/brands/georgiaDetail";
        } else if (brandId == 9) {
            return "cocacola/brands/minutemaideHome";
        } else {
            return "cocacola/brands/brandDetail";
        }
    }
}
```

기존 .do 방식의 request.getParameter() 대신
@RequestParam으로 파라미터를 바로 바인딩합니다

브랜드마다 화면 구성과 UX가 다르기 때문에
브랜드 ID 기준으로 View를 분기 처리했습니다.

brandId = 1: 코카콜라 제품 홈 + 제품 탭 구조

brandId = 8: 지오지아 상세 페이지

brandId = 9: 미닛메이드 전용 화면

그 외: 공통 브랜드 상세 화면

04. 주요 기능 구현 - Brands

```
package cocacola.mvc.mapper;

import java.util.List;

@Mapper
public interface BrandMapper {

    // 브랜드 전체 목록
    List<BrandDTO> selectBrandList();

    // 브랜드 상세
    BrandDTO selectBrand(int brandId);

    // 브랜드 영상
    BrandVideoDTO selectBrandVideo(int brandId);

    // 브랜드별 제품 목록
    List<ProductDTO> selectProducts(int brandId);

    // 서브 브랜드 목록
    List<SubBrandDTO> selectSubBrands(int brandId);
}
```

Mapper는 인터페이스만 정의하고
구현체는 MyBatis가 자동으로 생성합니다.

04. 주요 기능 구현 - Brands

```
package cocacola.mvc.domain;

import java.util.List;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class BrandDTO {

    private int brandId;
    private String brandName;
    private String brandLogoUrl;
    private String brandDescription;
    private String brandOrigin;
    private String brandMainphotoUrl;

    // 연관 객체 리스트
    private List<ProductDTO> products;
    private List<SubBrandDTO> subBrands;

    private String videoDescription;
    private String youtubeVideoId;
    private String brandCategories;
    private String videoThumbnailUrl;

    // brandCategories가 null일 경우 빈 문자열 반환하는 로직 유지
    public String getBrandCategories() {
        return brandCategories == null ? "" : brandCategories;
    }
}
```

BrandDTO는 브랜드 정보를 담는 데이터 전달 객체입니다.

04. 주요 기능 구현 - Brands

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper
  PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
  "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="cocacola.mvc.mapper.BrandMapper">

  <!-- 브랜드 전체 목록 -->
  <select id="selectBrandList" resultType="cocacola.mvc.domain.BrandDTO">
    SELECT b.brand_id as brandId,
           b.brand_name as brandName,
           b.brand_logo_url as brandLogoUrl,
           NVL(LISTAGG(LOWER(c.brand_category_name), ','), '')
             WITHIN GROUP (ORDER BY c.brand_category_name), '' AS brandCategories
    FROM Brands b
    LEFT JOIN Brand_Category bc ON b.brand_id = bc.brand_id
    LEFT JOIN Category c ON bc.brand_category_id = c.brand_category_id
    GROUP BY b.brand_id, b.brand_name, b.brand_logo_url
    ORDER BY b.brand_id
  </select>

  <!-- 브랜드 상세 정보 -->
  <select id="selectBrand" parameterType="int" resultType="cocacola.mvc.domain.BrandDTO">
    SELECT brand_id as brandId,
           brand_name as brandName,
           brand_logo_url as brandLogoUrl,
           brand_description as brandDescription,
           brand_origin as brandOrigin,
           brand_mainphoto_url as brandMainphotoUrl
    FROM Brands
    WHERE brand_id = #{brandId}
  </select>

  <!-- 브랜드 영상 정보 -->
  <select id="selectBrandVideo" parameterType="int" resultType="cocacola.mvc.domain.BrandVideoDTO">
    SELECT brand_id as brandId,
           youtube_video_id as youtubeVideoId,
           video_thumbnail_url as videoThumbnailUrl,
           video_description as videoDescription
    FROM Brand_Video
    WHERE brand_id = #{brandId}
  </select>

  <!-- 브랜드별 제품 목록 -->
  <select id="selectProducts" parameterType="int" resultType="cocacola.mvc.domain.ProductDTO">
    SELECT product_id as productId,
           brand_id as brandId,
           sub_brand_id as subBrandId,
           product_name as productName,
           product_volume as productVolume,
           product_image_url as productImageUrl
```

BrandMapper.xml은
브랜드 관련 모든 SQL을 담당하는 MyBatis 매퍼입니다.

04. 주요 기능 구현 - Our Company (Coca-cola Company, Since 1886)



비밀의 문이 열린다: 130여 년 역사를 간직한 기록보관소

코카-콜라 병의 원본 스케치부터 1950년대 코카-콜라 자판기, 88 서울 올림픽 성화봉까지.. 직원들도 허가 없이는 들어갈 수 없는 코카-콜라 기록보관소!

더 보기

코카-콜라 올림픽 파트너십의 역사

가장 오래된 올림픽 파트너인 코카-콜라. 1928년 암스테르담 올림픽부터 지금까지 약 한 세기 동안 쌓아온 코카-콜라의 올림픽 파트너십 역사를 살펴본다.

더 보기



더 많은 역사 이야기



코카-콜라 슬로건의 변천사

카피 한 줄로 소비자들의 마음을 사로잡은 비결은? "Drink, Coca-Cola"부터 "Real Magic"까지 코카-콜라 슬로건의 역사

더 보기 →



코카-콜라의 기발한 마케팅 전략들

최초의 무료 샘플링 쿠폰, 특허 받은 디자인.. 업계의 표준이 된 코카-콜라의 마케팅 전략들

더 보기 →



코카-콜라를 성장시킨 10가지 비즈니스 결정들

1886년 작은 약국에서 시작한 코카-콜라가 어떻게 세계적인 브랜드로 성장할 수 있었을까?

더 보기 →

[코카-콜라를 만든 사람들 제2편] 코카-콜라 회사를 만든 '아사 캔들러'

존 펨버턴(John Pemberton) 박사가 세상에 없던 음료를 만들었다면, 아사 캔들러(Asa Candler)는 코카-콜라가 전 세계인이 즐기는 음료로 될 수 있도록 초석을 다졌다.

더 보기



[코카-콜라를 만든 사람들 제3편] 코카-콜라를 60년간 이끈 CEO, 로버트 우드러프의 리더십

코카-콜라의 역사를 이야기할 때 로버트 우드러프를 빼놓고 이야기할 수 없다. 그는 60년간 회사를 이끌며, 코카-콜라를 글로벌 기업으로 성장시켰다.

더 보기



코카-콜라의 비밀: 제조법이 지금까지 공개되지 않은 비결은?

1886년 존 펨버턴 박사가 코카-콜라를 발명한 이후, 코카-콜라의 제조법은 철저히 비밀에 부쳐져 왔다.

더 보기



글로벌 코카-콜라

전 세계인들의 일상 속 빠짐없이 손님을 함께 해온 코카-콜라의 글로벌 비전과 가치, 지속 가능한 경영 전략

더 보기



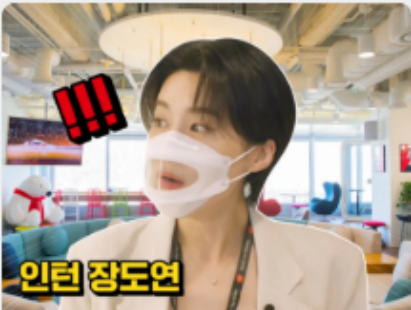
한국 코카-콜라

전 세계 200여 개국의 소비자들에게 상쾌함과 즐거움을 제공 하고 있는 코카-콜라. 한국에서 코카-콜라의 역사는 언제, 어떻게 시작되었을까?

더 보기



스페셜 기획 : 오피스 투어



한국 코카-콜라에는 특별한 기업문화가 있다?

명료에 대한 열정을 바탕으로 새로운 도전과 혁신을 만들어가는 코카-콜라의 기업문화를 알아보는 특별한 오피스 투어

더 보기 →



미국 애틀랜타 코카-콜라 본사&박물관 투어

코카-콜라 미국 본사와 월드 오브 코카-콜라(World of Coca-Cola) 박물관이 궁금하다면? 빠른편과 함께한 본적 코카-콜라 체험기!

더 보기 →

04. 주요 기능 구현 - Our Company (Since 1886)

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "https://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="cocacola.mvc.mapper.OurCompanyMapper">

    <select id="selectAllContents" resultType="cocacola.mvc.domain.OurCompanyDTO">
        SELECT
            CONTENT_ID    AS contentId,
            SECTION_CODE  AS sectionCode,
            TITLE,
            SUMMARY,
            IMAGE_FILENAME AS imageFilename,
            BUTTON_LINK    AS buttonLink,
            DISPLAY_ORDER  AS displayOrder
        FROM OURCOMPANY
        ORDER BY DISPLAY_ORDER ASC
    </select>

</mapper>
```

```
package cocacola.mvc.mapper;
```

```
import java.util.List;
import org.apache.ibatis.annotations.Mapper;
import cocacola.mvc.domain.OurCompanyDTO;
```

```
@Mapper // MyBatis Mapper 인터페이스임을 명시
public interface OurCompanyMapper {
```

```
    // 메서드 이름이 Mapper XML의 id와 일치해야 합니다.
    List<OurCompanyDTO> selectAllContents();
}
```

OurCompanyMapper (Java & XML 파일)

OurCompanyMapper.java 인터페이스의 메서드명과
XML의 id를 일치시켜 Runtime에 동적 프록시 객체가
생성되도록 설계했습니다.

SQL의 컬럼명을 자바의 필드명으로 Alias를 지정하여 DTO
객체에 자동 투영되도록 구성했습니다.

쿼리 결과는 DISPLAY_ORDER 순으로 정렬되어 데이터의
출력 순서를 보장합니다.

04. 주요 기능 구현 - Our Company (Since 1886)

```
<context:component-scan base-package="com.coke.play"></context:component-scan>

<bean id="hikariConfig" class="com.zaxxer.hikari.HikariConfig">
  <property name="driverClassName" value="net.sf.log4jdbc.sql.jdbcapi.DriverSpy"></property>
  <property name="jdbcUrl" value="jdbc:log4jdbc:oracle:thin:@localhost:1521:xe"></property>
  <property name="username" value="cola_user"></property>
  <property name="password" value="cola"></property>
</bean>

<bean id="dataSource" class="com.zaxxer.hikari.HikariDataSource" destroy-method="close">
  <constructor-arg ref="hikariConfig"></constructor-arg>
</bean>

<bean id="jdbcTemplate" class="org.springframework.jdbc.core.JdbcTemplate">
  <property name="dataSource" ref="dataSource" />
</bean>

<bean id="sqlSessionFactoryBean" class="org.mybatis.spring.SqlSessionFactoryBean">
  <property name="dataSource" ref="dataSource"></property>
  <property name="configLocation" value="classpath:com/coke/play/mapper/mybatis-
config.xml"></property>
  <property name="mapperLocations" value="classpath:cocacola/mvc/mapper/*.xml"></property>
</bean>

<mybatis-spring:scan base-package="cocacola.mvc.mapper"/>

<bean id="transactionManager"
class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
  <property name="dataSource" ref="dataSource" />
</bean>

</beans>
```

root-context.xml

HikariCP를 사용하여 OracleDB 연결을 관리합니다.
 SQL 세션을 생성하는 Factory 객체를 설정하고,
 실행할 SQL문이 담긴 XML 파일들의
 경로를 지정합니다.
 매퍼 인터페이스를 자동으로 스캔하여
 스프링 컨테이너가 관리하는 빈으로 등록함으로써
 코드의 의존성을 해결합니다.

04. 주요 기능 구현 - Discover (Brandstory, Coke&Meal, Originals)

기존 Servlet 방식

```
package cocacola.mvc.controller;

import java.io.IOException;

@WebServlet("/cocacola/Discover1/brandstory")
public class BrandStoryServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        BrandStoryDAO dao = new BrandStoryDAO();

        // 1 상세 페이지 처리 로직
        // 주소창에 ?id=숫자 파라미터가 넘어왔을 때 실행되
        String idParam = request.getParameter("id");
        if (idParam != null) {
            int id = Integer.parseInt(idParam);
            // 각 게시물마다 디자인이 다른 특성을 고려하여 ID별 개별 JSP 파일을 호출
            String jspPath =
                "/WEB-INF/views/cocacola/Discover1/brandStoryView_" + id + ".jsp";
            // 지정된 경로의 JSP로 제어권을 넘김
            request.getRequestDispatcher(jspPath).forward(request, response);
            return;
        }
    }
}
```

Spring Controller 방식

```
package cocacola.mvc.controller;

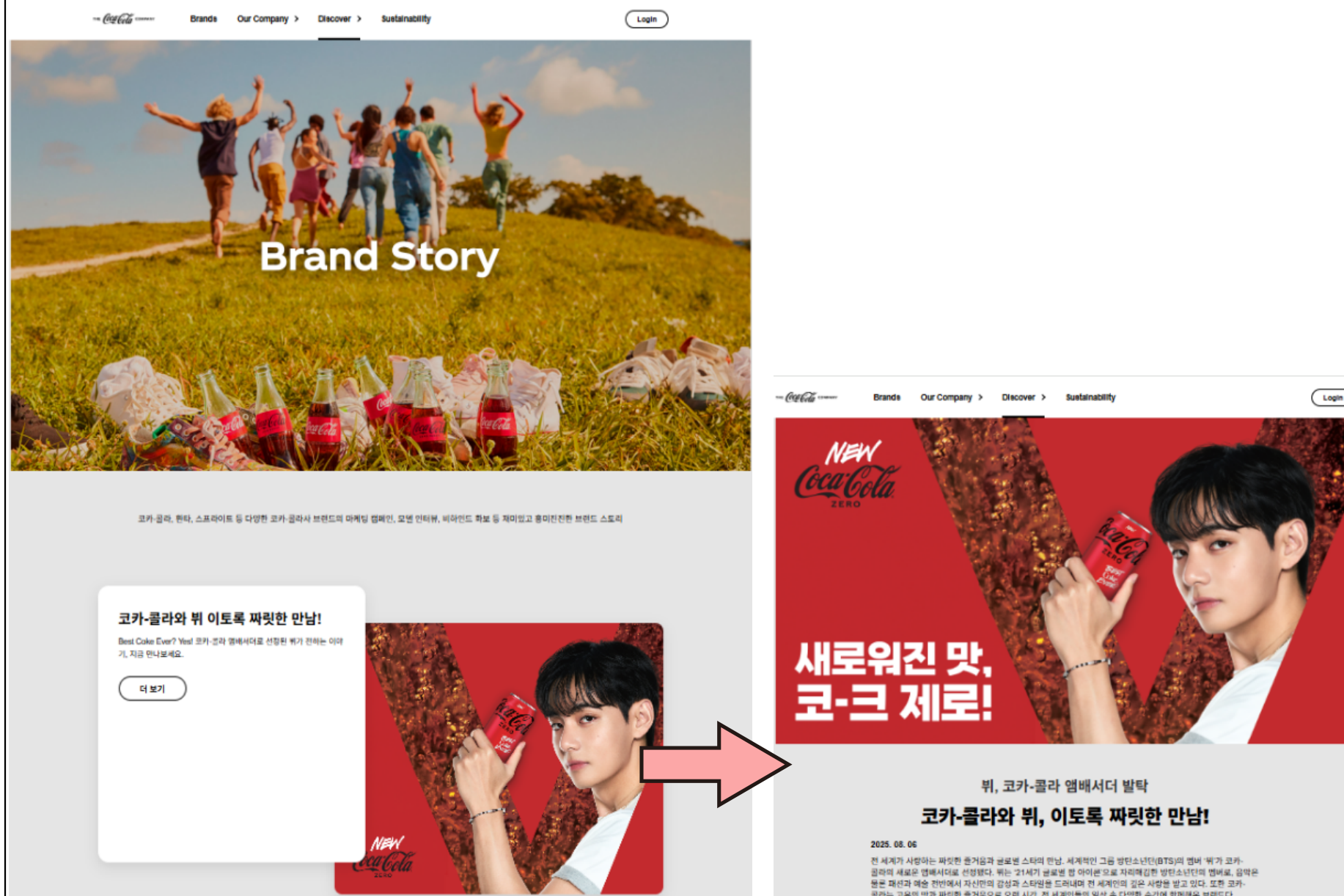
import java.sql.SQLException;

@Controller
@RequestMapping("/cocacola")
public class BrandStoryController {

    @Autowired
    private BrandStoryMapper brandStoryMapper;

    @GetMapping("/brandstory") // 요청 URL: /cocacola/brandstory
    public String brandStory(
        @RequestParam(value = "id", required = false) Integer id,
        Model model) throws SQLException {

        /* 1 상세 페이지 요청 처리 */
        if (id != null) {
            // id가 있을 때는 해당 상세 뷰 반환
            return "cocacola/Discover1/brandStoryView_" + id;
        }
    }
}
```



04. 주요 기능 구현 - Discover(Brandstory, Coke&Meal, Originals)

기존 DAO 방식

```
try {
    // 드라이버 로딩
    Class.forName("oracle.jdbc.driver.OracleDriver");
    // 연결 객체 생성
    conn = DriverManager.getConnection(url, user, password);

    // sql 순번대로 불러오기
    String sql = "SELECT * FROM BRAND_STORY ORDER BY brand_story_id ASC";

    pstmt = conn.prepareStatement(sql);
    rs = pstmt.executeQuery();

    while(rs.next()) {
        BrandStoryDTO dto = new BrandStoryDTO();
        dto.setId(rs.getInt("brand_story_id"));
        dto.setTitle(rs.getString("brand_story_title"));
        dto.setSummary(rs.getString("brand_story_summary"));
        dto.setImage(rs.getString("brand_story_image"));
        dto.setDate(rs.getDate("brand_story_date"));
        list.add(dto);
    }
} catch (Exception e) {
```

매퍼 XML 방식

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN" "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="cocacola.mvc.mapper.BrandStoryMapper">
    <select id="selectAll" resultType="cocacola.mvc.domain.BrandStoryDTO">
        SELECT
            brand_story_id AS id,
            brand_story_title AS title,
            brand_story_summary AS summary,
            brand_story_image AS image,
            brand_story_date AS "date"
        FROM BRAND_STORY
        ORDER BY brand_story_id ASC
    </select>

    <select id="selectById" parameterType="int" resultType="cocacola.mvc.domain.BrandStoryDTO">
        SELECT
            brand_story_id AS id,
            brand_story_title AS title,
            brand_story_summary AS summary,
            brand_story_image AS image,
            brand_story_date AS "date"
        FROM BRAND_STORY
        WHERE brand_story_id = #{id}
    </select>
</mapper>
```


04. 주요 기능 구현 - Discover(Opener, Creation, CokePlay APP)

```
@Controller
@RequestMapping("/Discover2")
public class OpenerController {

    @Autowired
    private OpenerService openerService;

    @Autowired
    private OpenerPostService postService;

    @GetMapping("/opener")
    public String opener(Model model) throws Exception {

        model.addAttribute("categoryList", openerService.getCategoryList());
        model.addAttribute("postList", postService.getPostListByCategory(1));
        model.addAttribute("postList2", postService.getPostListByCategory(2));
        model.addAttribute("postList3", postService.getPostListByCategory(3));

        return "cocacola/Discover2/opener";
    }
}
```

```
@Service
public class OpenerService {

    @Autowired
    private OpenerCategoryMapper openerCategoryMapper;

    public List<OpenerCategoryDTO> getCategoryList() throws Exception {
        return openerCategoryMapper.selectCategoryList();
    }
}
```

04. 주요 기능 구현 - Discover(Opener, Creation, CokePlay APP)

```
public interface OpenerPostMapper {  
    List<OpenerPostDTO> selectByCategory(int categoryId);  
}
```

```
<mapper namespace="cocacola.mvc.mapper.OpenerPostMapper">  
    <select id="selectByCategory"  
        parameterType="int"  
        resultType="OpenerPostDTO">  
        SELECT POST_ID, CATEGORY_ID, TITLE, SUMMARY,  
            THUMBNAIL_URL, EXTERNAL_URL, DETAIL_URL  
        FROM OPENER_POST  
        WHERE CATEGORY_ID = #{categoryId}  
        ORDER BY POST_ID  
    </select>  
</mapper>
```

04. 주요 기능 구현 - Sustainability

SustainabilityController.java

```
package cocacola.mvc.controller;

import com.yourcompany.mvc.model.vo.Document;

@Controller
@RequestMapping("/sustainability")
public class SustainabilityController {

    @Autowired
    private SustainabilityService sustainabilityService;

    @GetMapping
    public String sustainability(Model model) {

        List<Section> sections =
            sustainabilityService.getAllSections();

        List<Document> waterDocs =
            sustainabilityService.getDocumentsBySection(3);

        model.addAttribute("sections", sections);
        model.addAttribute("waterDocs", waterDocs);

        return "cocacola/Sustainability/sustainability";
    }
}
```

사용자 요청 → Service 호출
→ 데이터 조회 → View 전달

Sustainability 페이지의 전체 데이터를
준비하는 Controller

04. 주요 기능 구현 - Sustainability

```
package cocacola.mvc.service;
```

```
import java.util.List;[]
```

```
@Service
```

```
public class SustainabilityService {
```

```
    @Autowired
```

```
    private SustainabilityMapper sustainabilityMapper;
```

```
    /**
```

```
     * Sustainability 전체 섹션 조회
```

```
     */
```

```
    public List<Section> getAllSections() {
```

```
        return sustainabilityMapper.getAllSectionsWithMedia();
```

```
    }
```

```
    /**
```

```
     * 특정 섹션의 문서 조회
```

```
     */
```

```
    public List<Document> getDocumentsBySection(int sectionId) {
```

```
        return sustainabilityMapper.getDocumentsBySection(sectionId);
```

```
    }
```

```
    /**
```

```
     * Content 단건 조회 (Media 포함)
```

```
     */
```

```
    public Content getContentById(int contentId) {
```

```
        return sustainabilityMapper.getContentDetailById(contentId);
```

```
    }
```

```
}
```

SustainabilityService.java

필요한 데이터 로직을 메서드 단위로 분리

섹션, 문서, 상세 콘텐츠 조회 기능 제공

Controller는 Service만 호출하도록 구조 분리

04. 주요 기능 구현 - Sustainability

```
<mapper namespace="cocacola.mvc.mapper.SustainabilityMapper">
  <select id="getAllSectionsWithMedia" resultMap="sectionResultMap">
    SELECT * FROM SECTION ORDER BY section_id
  </select>

  <!-- Section -->
  <resultMap id="sectionResultMap" type="com.yourcompany.mvc.model.vo.Section">
    <id property="section_id" column="section_id"/>
    <result property="section_title" column="section_title"/>
    <result property="section_subtitle" column="section_subtitle"/>
    <result property="section_image" column="section_image"/>
    <result property="section_description" column="section_description"/>

    <collection property="contents"
      ofType="com.yourcompany.mvc.model.vo.Content"
      select="getContentsBySectionId"
      column="section_id"/>
  </resultMap>

  <!-- Content -->
  <select id="getContentsBySectionId" resultMap="contentResultMap">
    SELECT * FROM CONTENT
    WHERE section_id = #{section_id}
    ORDER BY content_id
  </select>

  <resultMap id="contentResultMap" type="com.yourcompany.mvc.model.vo.Content">
    <id property="content_id" column="content_id"/>
    <result property="section_id" column="section_id"/>
    <result property="content_title" column="content_title"/>
    <result property="content_body" column="content_body"/>
    <result property="content_category" column="content_category"/>
    <result property="content_link" column="content_link"/>
    <result property="detail_content_id" column="detail_content_id"/>

    <collection property="mediaList"
      ofType="com.yourcompany.mvc.model.vo.Media"
      select="getMediaByContentId"
      column="content_id"/>
  </resultMap>
```

```
<resultMap id="mediaResultMap" type="com.yourcompany.mvc.model.vo.Media">
  <id property="media_id" column="media_id"/>
  <result property="content_id" column="content_id"/>
  <result property="media_file" column="media_file"/>
</resultMap>

<!-- resultMap → resultMap 변경 -->
<select id="getMediaByContentId" resultMap="mediaResultMap">
  SELECT * FROM MEDIA
  WHERE content_id = #{value}
  ORDER BY media_id
</select>

<!-- ===== -->
<!-- 기타 -->
<!-- ===== -->

<select id="getDocumentsBySection" resultMap="documentResultMap">
  SELECT * FROM DOCUMENT
  WHERE section_id = #{sectionId}
  ORDER BY document_date DESC
</select>

<select id="getContentById" resultMap="contentResultMap">
  SELECT * FROM CONTENT
  WHERE content_id = #{contentId}
</select>

<select id="getContentDetailById" resultMap="contentWithMediaMap">
  SELECT
    c.*,
    m.media_id,
    m.media_file_type,
    m.media_file,
    m.media_description
  FROM CONTENT c
  LEFT JOIN MEDIA m
    ON c.content_id = m.content_id
  WHERE c.content_id = #{contentId}
  ORDER BY m.media_id
</select>
```

SustainabilityMapper.xml

04. 로그인/회원가입

TBL_MEMBER			
논리 이름*	도메인	데이터 타입	물리 이름*
 회원 ID	N/A	VARCHAR2(50)	USERID
 회원 비밀번호	N/A	VARCHAR2(100)	USERPW
 회원 이름	N/A	VARCHAR2(50)	USERNAME
 이메일	N/A	VARCHAR2(100)	EMAIL
 회원 가입일	N/A	DATE	REGDATE
 전화번호	N/A	VARCHAR2(20)	PHONE

04. 주요 기능 구현 - 로그인/회원가입

로그인

코카콜라와 함께하는 짜릿한 순간!

아이디

비밀번호

로그인

아이디 찾기

비밀번호 찾기

회원가입

package cocacola.mvc.controller;

import javax.servlet.http.HttpServletRequest;

import javax.servlet.http.HttpServletResponse;

import org.springframework.stereotype.Controller;

import org.springframework.web.servlet.mvc.annotation.annotation.AnnotationMethodMapping;

import org.springframework.web.servlet.mvc.support.RedirectAttributes;

public class LoginController {

@Autowired

private MemberMapper memberMapper;

@GetMapping("/login")

public String loginPage() {

// prefix(/WEB-INF/views/) + 리턴값 + suffix(.jsp)가 합쳐져서

// /WEB-INF/views/cocacola/Login.jsp 파일을 찾게 됩니다.

return "cocacola/Login";

}

@PostMapping("/login.do")

public String login(MemberDTO dto, HttpServletRequest request) {

HttpSession session = request.getSession();

MemberDTO result = memberMapper.login(dto);

if (result != null) {

// 로그인 성공 시 세션에 저장

session.setAttribute("member", result);

return "redirect:/cocacola/ourcompany?page=main";

} else {

// 로그인 실패 시

return "redirect:/cocacola/login?error=true";

}

}

@GetMapping("/logout.do")

public String logout(HttpSession session) {

session.invalidate(); // 세션 무효화

return "redirect:/cocacola/ourcompany?page=main";

}

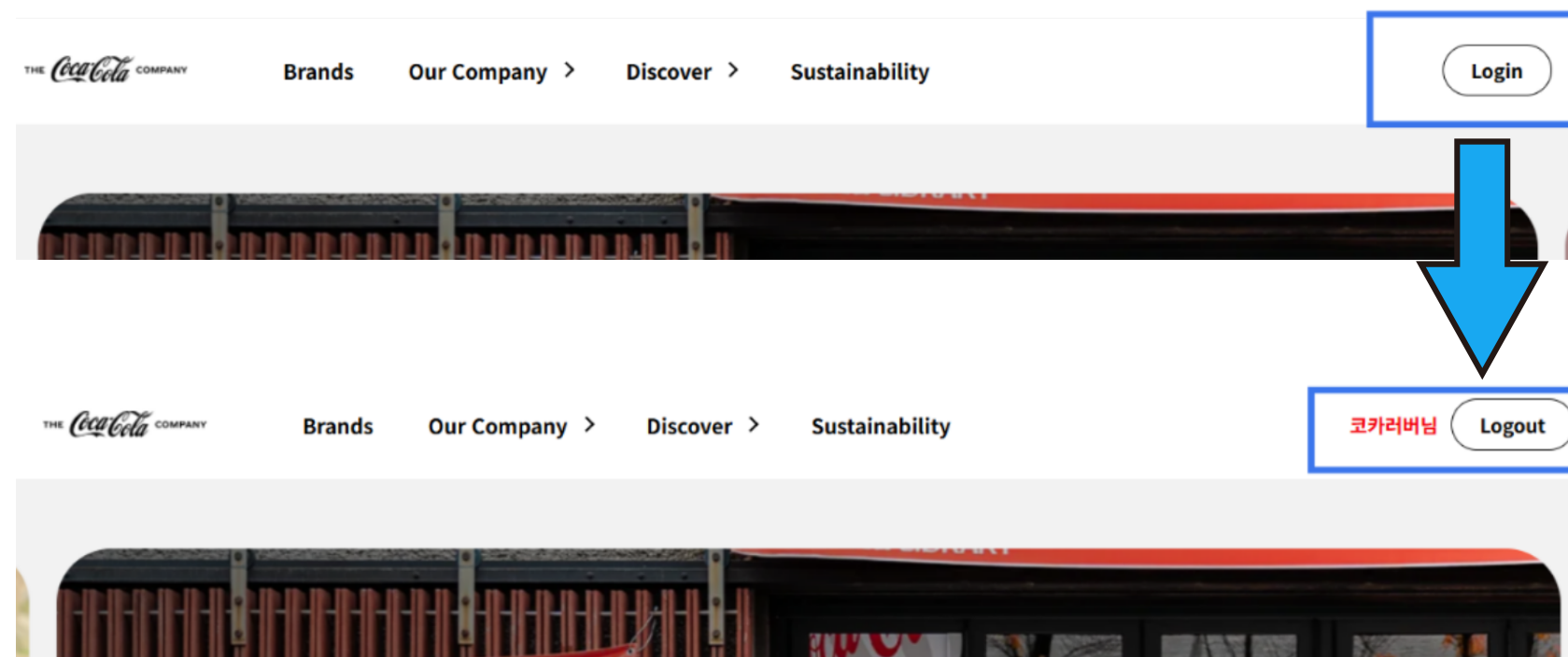
}

LoginController.java

MemberMapper.java

MemberMapper.xml

04. 주요 기능 구현 - 로그인/회원가입



`<c:choose>`

```

<!-- 1. 세션에 member가 아예 없는 경우: Login만 표시 -->
<c:when test="${empty sessionScope.member}">
  <a href="${pageContext.request.contextPath}/cocacola/login" class="auth-btn">Login</a>
</c:when>

<!-- 2. 세션에 member가 있는 경우: 사용자 이름 + Logout만 표시 -->
<c:otherwise>
  <span class="user-info">${sessionScope.member.username}님</span>
  <a href="${pageContext.request.contextPath}/cocacola/logout.do" class="auth-btn">Logout</a>
</c:otherwise>
</c:choose>

```

nav.java

04. 주요 기능 구현 - 로그인/회원가입

회원가입

코카-콜라 멤버십에 가입하고 다양한 혜택을 누리세요.

아이디

사용할 아이디를 입력하세요

중복 확인

비밀번호

비밀번호를 입력하세요

이름

이름을 입력하세요

이메일

example@coca-cola.com

전화번호

010-0000-0000

가입 완료

이미 계정이 있으신가요? [로그인하기](#)

아이디

GOGOMA

사용 가능한 아이디입니다.

join.java

```
<script>
$(document).ready(function() {
    let isIdChecked = false; // 중복 체크 여부 확인용 플래그

    $("#checkBtn").click(function() {
        var userid = $("#userid").val();

        if(userid == "") {
            alert("아이디를 입력해주세요.");
            return;
        }

        $.ajax({
            url: "${pageContext.request.contextPath}/cocacola/idCheck",
            type: "POST",
            data: { "userid": userid },
            success: function(data) {
                if(data == "0") {
                    $("#idMsg").text("사용 가능한 아이디입니다.").css("color", "blue");
                    isIdChecked = true;
                } else {
                    $("#idMsg").text("이미 사용 중인 아이디입니다.").css("color", "red");
                    $("#userid").val("");
                    isIdChecked = false;
                }
            }
        });
    });
});
```

중복 확인

MemberMapper.java

```
// 아이디 중복 체크
int idCheck(String userid);
```

MemberMapper.xml

```
<select id="idCheck" resultType="int">
    SELECT COUNT(*) FROM tbl_member WHERE userid = #{userid}
</select>
```

JoinController.java

```
@PostMapping("/idCheck")
@ResponseBody // 데이터를 페이지 이동 없이 브라우저로 직접 보냄
public String idCheck(String userid) {
    int count = memberMapper.idCheck(userid);
    return String.valueOf(count); // "0"이면 사용 가능, "1"이면 중복
}
```

04. 주요 기능 구현 - 로그인/회원가입

회원가입

코카-콜라 멤버십에 가입하고 다양한 혜택을 누리세요.

아이디

사용할 아이디를 입력하세요

중복 확인

비밀번호

비밀번호를 입력하세요

이름

이름을 입력하세요

이메일

example@coca-cola.com

전화번호

010-0000-0000

가입 완료

이미 계정이 있으신가요? [로그인하기](#)

MemberMapper.java

```
// 회원가입  
void insertMember(MemberDTO dto);
```

MemberMapper.xml

```
<insert id="insertMember"  
    parameterType="cocacola.mvc.domain.MemberDTO">  
    INSERT INTO tbl_member (userid, userpw, username, email, phone)  
    VALUES ({userid}, {userpw}, {username}, {email}, {phone})  
</insert>
```

JoinController.java

```
// 회원가입 데이터 처리  
@PostMapping("/join.do")  
public String join(MemberDTO dto, RedirectAttributes rttr) {  
    // DB의 member 테이블에 저장하는 메서드 호출  
    memberMapper.insertMember(dto);  
  
    // "userName"이라는 이름으로 사용자의 실명을 담아 보냅니다.  
    rttr.addFlashAttribute("userName", dto.getUsername());  
  
    // 가입 완료 후 로그인 페이지로 보냄  
    return "redirect:/cocacola/login";  
}
```

04. 주요 기능 구현 - 아이디/비밀번호 찾기

```
@Controller
@RequestMapping("/cocacola")
public class FindIdController {

    @Autowired
    private MemberMapper memberMapper;

    // 아이디 찾기 페이지 이동
    @GetMapping("/FindId")
    public String findIdPage() {
        return "cocacola/FindId"; // JSP 이름
    }

    // 아이디 찾기 처리
    @PostMapping("/FindId")
    public String findId(MemberDTO dto, Model model) {

        // 입력 전화번호에서 하이픈 제거
        if(dto.getPhone() != null) {
            dto.setPhone(dto.getPhone().replace("-", ""));
        }

        // 전화번호로 회원 조회
        MemberDTO member = memberMapper.findByPhone(dto);

        if (member == null) {
            model.addAttribute("msg", "등록된 전화번호가 없습니다.");
            model.addAttribute("success", false);
            return "cocacola/FindId";
        }

        model.addAttribute("msg", "회원님의 아이디는 " + member.getUserid() + " 입니다.");
        model.addAttribute("success", true);

        return "cocacola/FindId";
    }
}
```

```
@Controller
@RequestMapping("/cocacola")
public class PasswordController {

    @Autowired
    private MemberMapper memberMapper;

    // 비밀번호 찾기 페이지 (GET)
    @GetMapping("/findPass")
    public String findPasswordPage() {
        return "cocacola/FindPass"; // JSP 파일 이름
    }

    // 비밀번호 찾기 처리 (POST)
    @PostMapping("/findPass")
    public String findPassword(MemberDTO dto, Model model) {

        MemberDTO member = memberMapper.findByIdAndEmail(dto);

        if (member == null) {
            model.addAttribute("msg", "아이디 또는 이메일이 일치하지 않습니다.");
            model.addAttribute("success", false);
            return "cocacola/FindPass";
        }

        model.addAttribute("msg", "비밀번호는 " + member.getUserpw() + " 입니다.");
        model.addAttribute("success", true);

        return "cocacola/FindPass";
    }
}
```

아이디와 비밀번호 찾기는 Spring MVC에서 GET/POST를 분리해
폼 입력 → DB 조회 → 결과 메시지 출력 흐름으로 구현했습니다.

05. 시연



06. 후기



오승찬

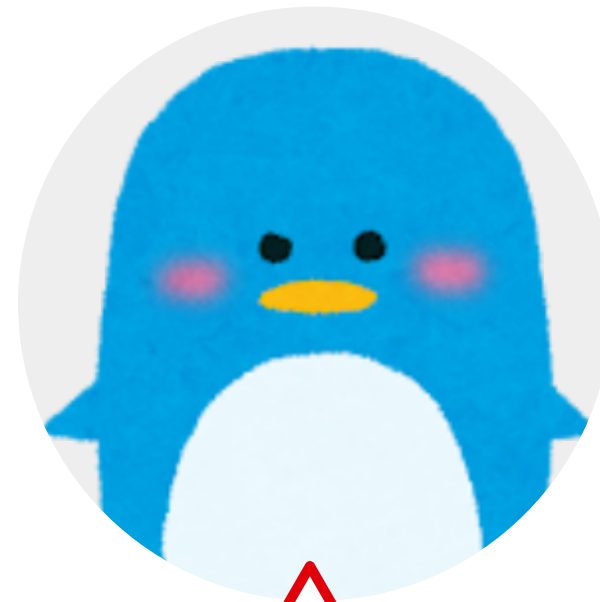
팀장

스프링 구조로 변환하는 과정에서 스프링에 대해 좀 더 잘 알게 되었고 앞으로 할 프로젝트에 대해서도 감을 잡을 수 있어서 뜻깊었습니다.



김다원

기존에 구현했던 기능을 Spring 구조로 다시 설계하면서 컨트롤러, 매퍼, SQL 분리 구조에 대한 감을 잡을 수 있었던 프로젝트였습니다



김수진

스프링 MVC 구조를 직접 사용해 보면서 각 역할을 나누는 이유를 자연스럽게 이해하게 되었고, 처음엔 복잡해 보였지만 구조를 잡아두니 코드 흐름이 훨씬 명확해졌습니다



이나영

MVC 패턴을 직접 사용해볼 수 있어서 좋았습니다.



장소예

지난 프로젝트 때 했던 길고 복잡했던 코드들이 스프링에서는 어노테이션을 쓰면 쉽고 간결해 진다는 것을 몸소 체감하여 스프링 프레임워크의 강력함을 직접 느낄 수 있었습니다.